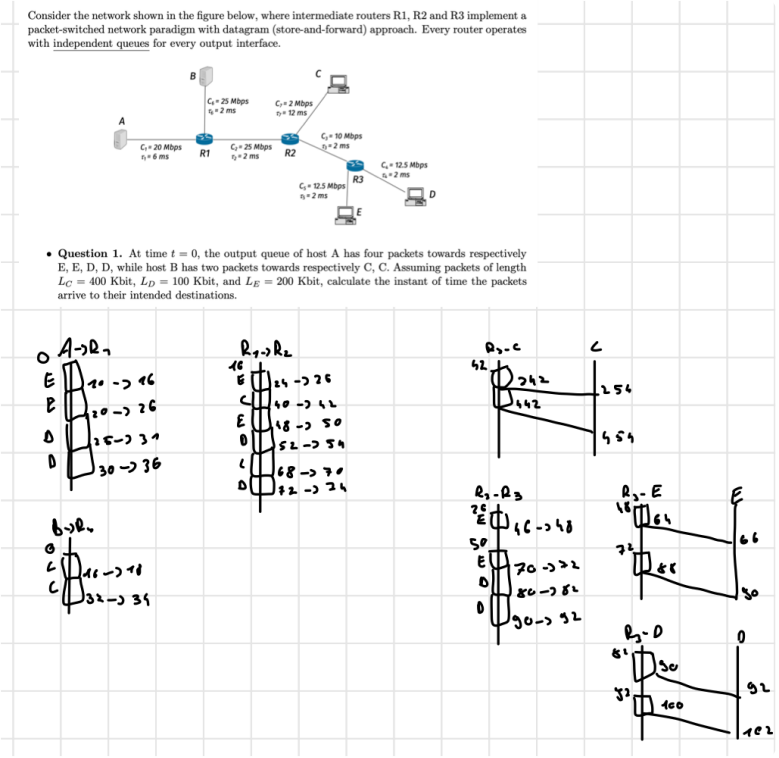
1) DLL

In a link, given **C** and **L** the **time to start to transmit all the message** is $T = L/C$

Packet Switched Networks

This exercise is only tedious, not hard.

- Draw one line for every interface and it's destination. Start from the source then move to the destination
- On each link calculate, starting from the first packet, the arrival time and the transmission time. Then also add the propagation time separately
  - If a packet arrives while a packet is still being transmitted it is queued and will have arrival time = propagation time of previous
- Remember that often different links have different queues



Example

UDP

Here you just:

- find the bottleneck
- find how many packets are sent k
- $$T = \sum_{i \neq b}(T_i + \tau_i) + kT_b + \tau_b = \sum_{i}(T_i + \tau_i) + (k-1)T_b$$

> **Remark.**
> UDP doesn't offer ACK so the time is just until the reception of the last packet

GBN ARQ

Find number of complete windows N_C and the number of packets in the last window N_L

$$N_C = \lfloor \frac{k}{N} \rfloor$$
$$N_L = k - N_c \cdot N$$

On every link:

- check $\forall$ links if they offer continuous transmission

$$N \cdot T_i \geq RTT = T_i + 2\tau_i$$

- If one or more links don't satisfy this we must:
  - In non bottleneck links we can send continuously
  - The packets will queue at the start of bottleneck link. Here there will be an idle time between the windows. This can be found by **finding the difference of time of the transmission of the last packet and the arrival of the ack of the first packet**

$$\Delta t_{\text{idle}} = T_b + 2\tau_b - NT_b$$

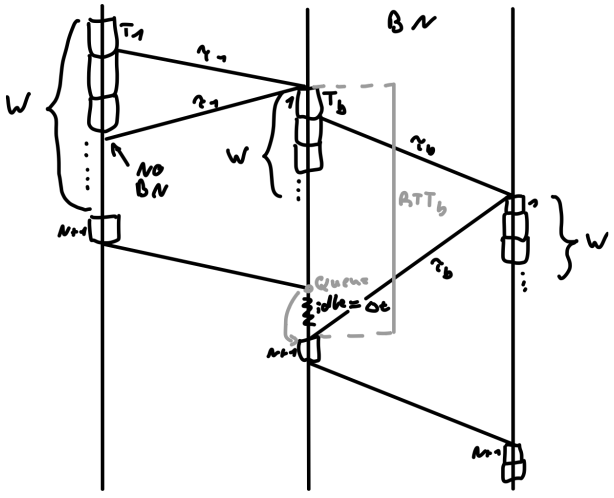
  - The time to send a complete window through the bottleneck link is $RTT_b = T_b + 2\tau_b$

$$T = \sum_i (T_i + \tau_i) + N_C \cdot RTT_b + (N_L - 1) \cdot T_b \ (+ACK \text{ if needed})$$

  - $$\overset{\text{ACK}}{\rightarrow} \sum_i (T_i + 2\tau_i) + N_C \cdot RTT_b + (N_L - 1) \cdot T_b$$

- If all links are continuous it behaves like UDP but **we must add ack**

$$T = T_{UDP} + ACK = T_{UDP} + \sum_{\text{all links}} \tau_i = \sum_i (T_i + 2\tau_i) + (k-1)T_b$$



Non Continuous GBN For All Links Example

End To End:

- Check with bottleneck link if it allows for continuous stream:

$$N \cdot T_b \geq RTT_{e2e} = \sum (T_i + 2\tau_i)$$

- If this is not satisfied we have
  - each window is sent every RTT_{e2e}
  - The idle time is given by

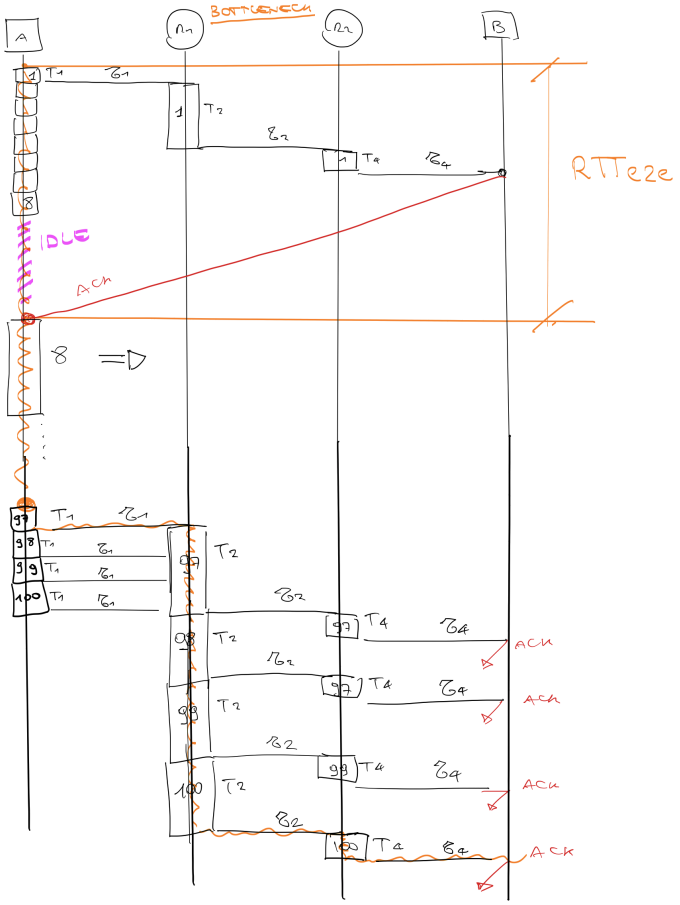
$$\Delta t_{\text{idle}} = RTT_{e2e} - N \cdot T_1$$

$$T = \sum_i (T_i + \tau_i) + N_C \cdot RTT_{e2e} + (N_L - 1) \cdot T_b \ (+ACK \text{ if needed})$$

$$\overset{\text{ACK}}{\rightarrow} \sum_i (T_i + 2\tau_i) + N_C \cdot RTT_{e2e} + (N_L - 1) \cdot T_b = (N_c + 1) RTT_{e2e} + (N_L - 1) \cdot T_b$$

- If all links are continuous it behaves like UDP but **we must add ack**

$$T = T_{UDP} + ACK = T_{UDP} + \sum_{\text{all links}} \tau_i = \sum_i (T_i + 2\tau_i) + (k-1)T_b = RTT_{e2e} + (k-1)T_b$$



Non Continuous GBN E2E Example

Remark.
Clearly if the idle time is negative then there is NO idle time and the link is not a bottleneck

Remark (TO)**.**

The minimum TO is of RTT. Each TO timer starts when a packet is sent. If in the n-th window, the k-th packet is lost the TO will end at

$$T^* = (n-1)RTT + (k-1)T_1$$

S&W ARQ

On each link:

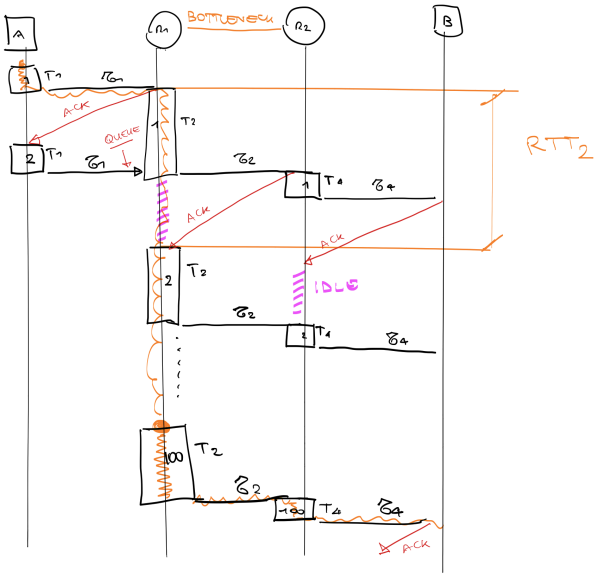
Here each packet get's sent upon receiving the ack of the previous one. The bottleneck link will queue packets. This is easily seen by graphically drawing the first few packets and their times. Therefore the main bottleneck will be given by

$$RTT_b = T_b + 2\tau_b = \max\{T_i + 2\tau_i\}$$

And the time will be

$$T = \sum_i (T_i + \tau_i) + (k-1)RTT_b \ (+ACK \text{ if needed})$$

$$\overset{ACK}{\rightarrow} \sum_{\text{all links}} (T_i + 2\tau_i) + (k-1)RTT_b$$

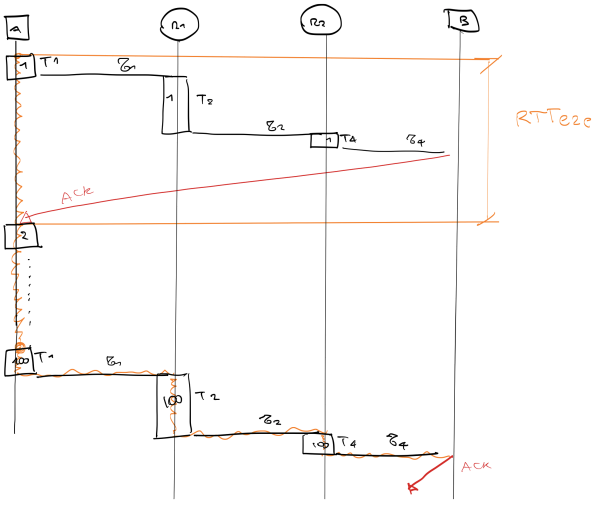


S&W On Each Link Example

End To End

Here each packet gets sent after the ack on the last link is received back. This will simply be

$$T = k \cdot RTT_{e2e} = k \cdot \sum_{\text{all links}} (T_i + 2\tau_i)$$



S&W E2E Example

Mix Between Links With GBN and S&W ARQ

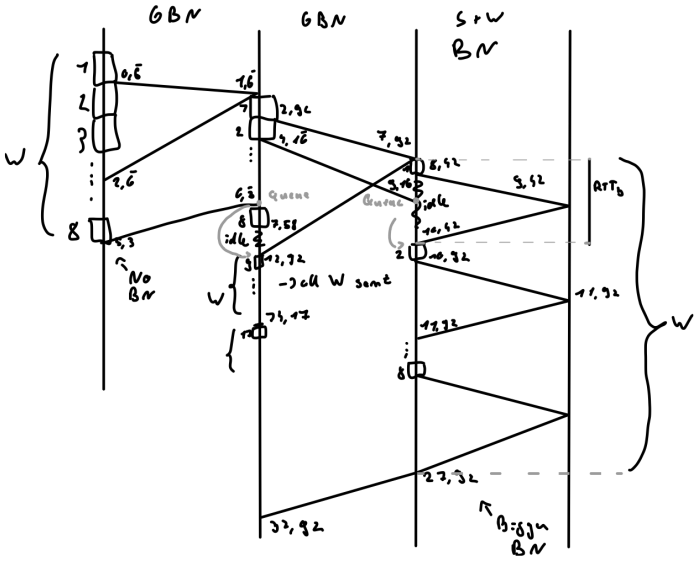
I believe that this section is better done ad hoc for every exercise by graphically drawing the packets and understanding where the bottlenecks are located and how the transmission behaves.

To find the bottleneck we find how much time a window takes to be sent (even on S&W)

- GBN: N packets are sent in $T + 2\tau$ if there is no bottleneck
- S&W: N packets are sent in $N \cdot RTT = N(T + 2\tau_i)$
  The slowest one is the bottleneck

In generale we can use the rules found in the previous chapters:

- GBN: behaves either like UDP or if the link is bottlenecked the packets will queue at the end
- S&W: can be seen as e2e on each link that has it



Example GBN,GBN,S&W

In fact it turns out that even though the queue happens in 2 places only the last link is the bottleneck. Numerically we obtained that:

$$\begin{aligned} \text{GBN}_1: &\quad T_1 + 2\tau_1 = 2.66\ [ms] \\ \text{GBN}_2: &\quad T_2 + 2\tau_2 = 11.25\ [ms] \\ \text{S\&W}_3: &\quad N(T_3 + 2\tau_3) = 20\ [ms] \rightarrow \text{bottleneck} \end{aligned}$$

If S&W is bottleneck (all packets queue at start of this link)

$$T = \sum_i (T_i + \tau_i) + (k-1)(T_b + 2\tau_b)\ (+ACK \text{ if needed})$$

If GBN is bottleneck (all packets queue at start of this link)

$$T = \sum_i (T_i + \tau_i) + N_C \cdot RTT_b + (N_L - 1) \cdot T_b\ (+ACK \text{ if needed})$$

TCP

Recall that the time to send one MSS is $T_{packet} = L/C\ [s]$ and for one bit it is $T = T_{bit} = 1/C\ [s]$.

With TCP we have:

$$\begin{aligned} BDP = R_{net} \cdot RTT & = R_{bottlenexck} \cdot RTT [bits] \\ & = \frac{R_{bn} \cdot RTT}{1\ MSS} [pkts] \\ \text{Throughput} = \frac{cwnd_{\max}}{RTT} & \end{aligned}$$

Setup times:

$$\begin{aligned} \text{No Piggyback:} \quad & T_{setup} = 2 \cdot \sum \tau_i \\ \text{No Piggyback:} \quad & T_{setup} = (SYN) + (ACK + SYN) + (ACK + PIGGY) \\ & = \sum T_i + 3 \cdot \sum \tau_i \end{aligned}$$

Now let's go to the **send time, here T_{setup} is always with no piggyback assumed since it is already counted in 1st transmission:**

Claim (Intuition behind the formula)**.**

- ***Non Continuous Flow:***

$$T = T_{setup} + s \cdot RTT + (c-1)T_{bottleneck} + \beta$$

  - *s: is the number of windows sent*
  - *c: is the packets sent in the last window (minus 1)*
  - *β is:*
    - *0 if we want last ack*
    - *$\sum(T_b + \tau_i) - RTT$ if we want last bit (we see last window as UDP)*
- ***Continuous Flow:***

$$T = T_{setup} + s \cdot RTT + (c-1)T_{bottleneck} + \alpha$$

  - *s: is the number of windows before cf was reached*
  - *c: is the packets sent in cf (minus 1)*
  - *α is*
    - *RTT if we want the final ack*
    - *$\sum(T_i + \tau_i)$ if we want reception of last byte*
  - *The final α is due to the fact that cf is "one big last window", that is now coherent with the non continuous case*

1. Find the amount of TCP MSS packets to send and also the continuous flow condition:

$$\text{swnd} \cdot T_{bottleneck} \geq RTT \rightarrow \text{swnd} \geq \frac{RTT}{T_{bottleneck}} = RTT \frac{R}{L = 1\ MSS} = \lceil BDP \rceil$$

2. **SS**: see how many windows s it takes to reach sshthresh and calculate how many packets were sent:

$$s = \lfloor \log_2(sshthresh) \rfloor$$
$$p = \sum_{i=0}^{s-1} (2^i) = 2^s - 1$$

   in some rare cases it is possible that we already sent enough packets before reaching sshthresh.

- **CS**: we set $\lceil BDP \rceil = C$, now calculate **(if rwnd is negligible)**. C is the value of cwnd where we stop (last we send C-1 pkts max)

$$s' = C - 2^s$$
$$p' = \left(\sum_{i=p}^{C-1} i \right) = \frac{(C-p)(C+p-1)}{2}$$
$$\text{In total have sent p' packets in s+s' windows}$$

- If $p' \leq M$ we reach continuous flow Now the remaining $c = M - (p')$ packets are sent in continuous flow

$$T = T_{setup} + (s+s')RTT + (c-1)T_{bottleneck} + \alpha$$

- If $p' \geq M$ continuous flow is **never reached** and thus we send all packets in CS. To solve this find the solution to $p'(C) = M$ for C (often very easy just count, otherwise 2nd deg equation). This is the swnd stop point. The last window sent a total of c packets (find graphically)

$$T = T_{setup} + (s+s')RTT + (c-1)T_{bottleneck} + \beta$$

Remark.

IF CS is not reached C is found as

$$C = \frac{1 + \sqrt{1 + 8M + 4p(p-1)}}{2}$$

If $C \in \mathbb{N}$ then you can redo the CC step normally.

If $C \notin \mathbb{N}$ then take the floor $C \leftarrow \lfloor C \rfloor$ and compute CC now we have the following:

- $s + s' + 1$ windows ($s + s'$ **complete**)
- $p' \leq M$ and therefore the last window will have $c = M - p'$ packets

Remark.

In general to find the time just expand as much as possible the calculations

- In CF we are limited by BN:

$$T = T_{set} + (s+s')RTT + T_1 + \\ + \tau_1 + \ldots + cT_b + \tau_b + \ldots \text{ here all arrived} + \sum \tau_i \text{ here ACK arrived}$$

- In CC we have:

$$T = T_{set} + s_{complete}RTT + \text{ here all complete windows are sent} \\ \overset{\text{ACK}}{\rightarrow} + RTT + (c-1)T_b \text{ here we just count in windows} \\ \overset{\text{last bit}}{\rightarrow} + (c-1)T_b + \sum (T_i + \tau_i) \text{ here last window is seen as UDP}$$

Remark.

If we are CC and send exactly s windows $N_L = 0$ we have

- Last ack: s=s and c=cwnd

- Last bit: s=s-1 and c=cwnd

Error in Transmission

In general if there is one error in the transmission we split the problem in 2 parts; before and after the error. The second part is seen as an errorless transmission with $M' = M - p_1'$ packets the time starting with an offset $T = T_2 + TO + T_1 = T_2 + T'$. Two main cases occur:

- **We loose all in flight packets** of the n-th window.

$$T' = T_{setup} + (n-1)RTT + TO$$

- **We loose a segment instead of a window** we must consider the TO to start when the segment is sent. Segment lost in window n at position k. Then the to starts in that window at time

$$(k-1)T_1$$

  And the time before the TO becomes

$$T = T_{setup} + (n-1)RTT + (k-1)T_1 + TO$$

Based on what type of TCP we have, different TO conditions occurr

- In **Fast RETX** after a TO we have **ssh=cwnd/2** and **cwnd=1**

But in TCP Reno and later maybe TO isn't even waited for (see theory part)

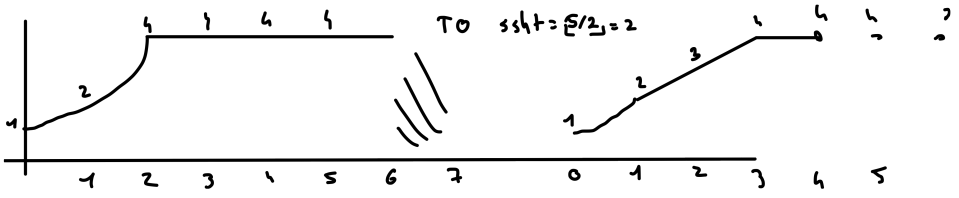
RWND non negligible

If **rwnd is non negligible** redo point **CS** with $C =$ rwnd to find how many packets were sent until that point. Three cases occurr:

- **rwnd is not reached**: same as before
- **rwnd is reached but after CF threshold**: same as when CF is reached
- **rwnd is reached and limits the connection**

If the last case happens then we just find how many packets and windows were sent until rwnd was hit. The calculate N_C and N_L and then the time is calculated similarly to the case where we're stuck in CC. Just with $c = N_L$ and s the number of windows until rwnd was reached $+N_L$

Example



Example

This shows 2 areas due to a TO at the 7-th window:

- Recall that the n-th window ends where n is on the x axis
- In the first part SS is shown, then we are limited to rwnd=4
- **After the TO sshthresh is halved (floor)**
- therefore in second part SS goes only until 2, then CC and then at swnd=rwnd we are again limited

Calculations:

First Part:

-SS: $\begin{cases} s = 2 \\ p = 3 \end{cases}$

-No CC:

-Until 7-th window we send another $4 \cdot 4 = 16$ pkts

In total: $\begin{cases} s = 6 \\ p = 19 \end{cases} \rightarrow M' = 21$

Second Part:

-SS: $\begin{cases} s'_{SS} = 1 \\ p'_{SS} = 1 \end{cases}$

-CC: $\{ s'_{CC} = 4 - 2^1 = 2$

-Only $M'' = M' - p' = 15$ pkts left

this is 3 windows + 3 pkts

In total: $\{ s' = s'_{SS} + s'_{CC} + 3 = 6$

Finally we have:

$$\begin{aligned} T &= T_{set} + T_1 + TO + T_2 + \alpha \\ &= T_{set} + s \cdot RTT + TO + (s'_{SS} + s'_{CC} + N_C) \cdot RTT + (N_L - 1) \cdot T_b + \gamma \end{aligned}$$

Keep an eye on γ since it might be very different in many cases, it depends on

- final transmission condition (SS, CS, CF, limited)
- ack or last byte

For CC and CF this second part is easily seen as an errorless TCP

- **CF**
  - $\gamma = \alpha$ as stated before
- **CC**
  - $\gamma = \beta$ as stated before
- **limited**
  - $\gamma = \beta$

> **Remark** (γ MIGHT BE WRONG)**.**

Estimate C, τ from Pin

A ping has this formula:

$$RTT = 2\left(\sum(T_i + \tau_i)\right)$$

with T_i, τ_i the parameters of the links the ping goes through. One value of I will be our two values to find. Once you have 2 RTT just solve the system.

> **Remark.**
>
> Recall that everything must have the same unit of measure so if L is in bytes write it in bits and C in bps (not Mbps) and the term 10^3 is to put it in ms
>
> $$T_i = \frac{L \cdot 8}{C} \cdot 10^3$$

Packet Size

LL_header_size = 36 bytes
This means that TCP pkts are encapsulated into LL ones (1 TCP pkt encapsulated into 1 LL pkt, by adding LL headers)
LL-pkt-size = LL-hdr-size + TCP-seg-size

While the payload is:
TCP_payload = TCP_seg_size - IP_hdr - TCP_hdr

Throughout

E2E:

$$\eta_{\max} = \frac{W_{\max}}{RTT} \text{ with: } RTT = \sum RTT_i$$

and then:

$$\boxed{B \approx \min\left\{\frac{W_{\max}}{RTT}, \frac{1}{RTT\sqrt{\frac{2bp}{3}} + T_o \min\{1, 3\sqrt{\frac{3bp}{8}}\}p(1+2p+4p^2)}\right\}}$$

finally

$$\eta = B \times \text{TCP\_payload}$$

in **bits**

at the app layer we have **TCP_payload-APP_hdr_size**

- Calculate the TCP throughput of connection B→D
- Links L2→L3→L7→L5

$$\texttt{RTT}_{\text{BD}} = \texttt{RTT}_2 + \texttt{RTT}_3 + \texttt{RTT}_5 + \texttt{RTT}_7 = 25 \text{ ms}$$

$$\sigma^2_{\text{BD}} = \sigma^2_2 + \sigma^2_3 + \sigma^2_5 + \sigma^5_7 =$$
$$= 0.1^2 + 2^2 + 1^2 + 0.2^2 = 5.05 \text{ ms}^2$$

$$\text{MAD}_{\text{BD}} \approx 0.797\sigma_{\text{BD}} = 1.791 \text{ ms}$$

$$\texttt{RTO} = \texttt{RTT}_{\text{BD}} + 4 * \texttt{MAD}_{\text{BD}}$$
$$= 25 + 4 * 1.791 = 32.164 \text{ ms}$$

$$T_o = \max\{\texttt{RTO}, 1 \text{ s}\} = 1 \text{ s}$$

Example

When more than one connection is active at a time:
Find the allocation rate of each link

$$\xi_i = \frac{b_i}{\sum b_j}$$

with $\sum b_j = B$ and $\sum \xi_i = 1$ and thus $b_i = B \cdot \xi_i$
For each connection the throughput is

$$\eta_i = \min\{\eta_{i_{E2E}}, b_i\}$$

Packet Error Rate

The error over one link is:
We compute the **error rate for a single TX** of a LL pkt, as

$$\tilde{p} = 1 - (1 - P_{bit})^L$$

Where (LL encapsulation) : **L=(LL_hdr+TCP_seg_size)**$\times$ **8**

The LL **"residual" pkt error rate** is

$$p = (\tilde{p})^M$$

The E2E pkt error rate from two points is:

$$p_{E2E} = 1 - \sum(1 - p_i)$$

BUT actually if $p_{E2E} = 0$ then the slowest link is bottleneck

RETX TO

Variance over path:

$$\sigma^2_{E2E} = \sum \sigma^2_i$$

from here

$$MAD_{E2E} = \sigma_{E2E}\sqrt{\frac{2}{\pi}} \approx 0.798\sigma_{E2E}$$

From here the **retx timeout is estimated as**

$$RTO = RTT_{E2E} + 4 \cdot MAD_{AE}$$

And finally the **timeout timer is:**

$$T_0 = \min\{RTO, 1\ [s]\}$$

1.2) Theory

TCP "Old" Tahoe

Implements:

- Slow Start
- Congestion Avoidance
- Error recovery only with time out: it RETX from first unACK pkt

TCP Tahoe (Fast Retx)

Implements:

- Slow Start
- Congestion Avoidance
- Fast Retransmit (FR)

FR works by sending the missing segment given by dupAck and then restarting the connection (SS, cwnd=1, ssthresh W/2).

TCP Reno (Fast Recovery)

Setting cwnd=1 is too aggressive, we advance cwnd by sshthrehs+ the number of dupacks recieved (does not violate pkt conservation). In Reno we enter **Fast Recovery** at the third dup ack:

- If we get 3 dupACKs for segment *N*
  - Retransmit (RETX) segment *N*
  - Set `ssthresh` = `cwnd` / 2
  - Set `cwnd` = `ssthresh` + 3
- For every subsequent dupACK
  - Increase `cwnd` by 1 MSS (as one pkt went through)
- When a new ACK is received
  - Reset `cwnd` to `ssthresh`
  - Resume connection in congestion avoidance

Pseudocode

When the third dupACK arrives
enter Fast Recovery:
`ssthresh` = `cwnd` / 2
RETX the missing segment **(Fast RETX)**
`cwnd` = `ssthresh` + 3 MSS
Every time a new dupACK arrives
`cwnd` = `cwnd` + 1 MSS
TX a packet if allowed by `cwnd`
When an ACK acknowledging new data arrives
`cwnd` = `ssthresh` (from first step above)
Exit Fast Recovery (resume normal operation in CA)

Pseudocode With K =3 dup acks

This means that we set the new sshtresh and we retx the missing frame. Then we advance the window by K dup akcs received. We then send the new packets (and advance windows) and receive dupacks (since we have halved W) Finally after sending all missing data we start from CA

4.2.3) TCP New Reno

This implements **Multiple RETX Fast Recovery**

```
When the Kth dupACK arrives
    recover  = lastseqno                          // highest TXed seq.no
    ssthresh = max (flightsize/2, 2 MSS)          // flightsize=lastseqno-ack_no+1
    cwnd  = ssthresh + 3 MSS                      // at least 3 OOO pkts RXed OK
    Reset RETX timer                              // to prevent timeout during fast recovery
    RETX the missing segment                      // Fast RETX
    TX a segment if allowed by cwnd
Every time a new dupACK arrives
    cwnd  = cwnd + 1 MSS                          // as 1 packet reached the receiver
    TX a packet if allowed by cwnd
When an ACK that acknowledges new data arrives
    if (ack_no>recover)
        cwnd = ssthresh                           // in step 1 above
        Exit from Fast Recovery
    else this is a "partial new ACK"              // acking "n_acked" packets
        cwnd  = cwnd - n_acked + 1
        Reset RETX timer                          // if Slow-but-steady TCP version
        RETX a packet with seq_no=ack_no
        TX a packet if allowed by cwnd
```

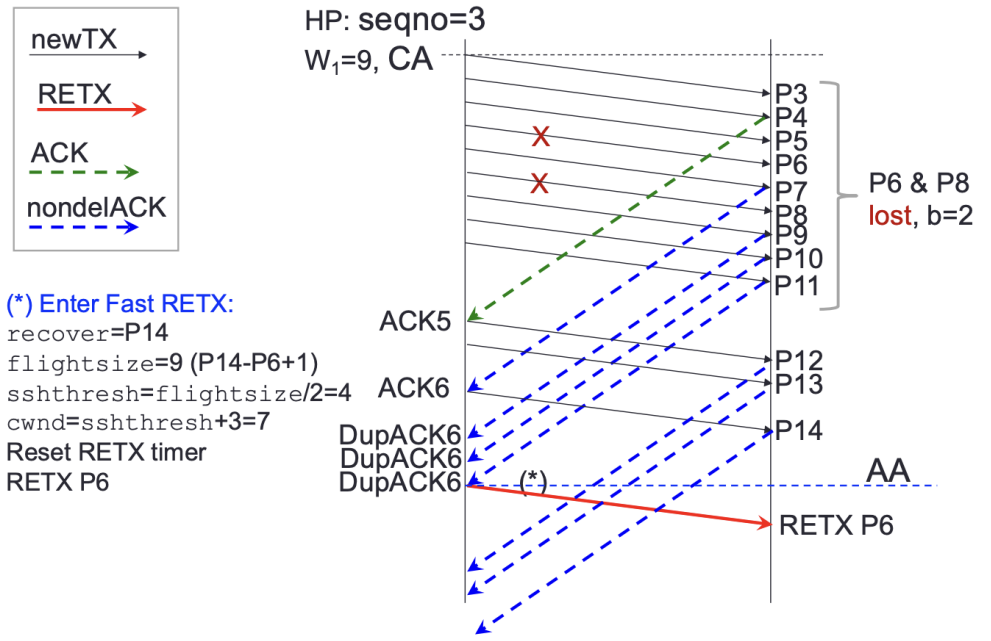
Pseudocode with K=3

4.2.3) TCP New Reno with Selective ACK

```
When the K-th dupACK arrives:
    recover = lastseqno;                          // highest TX seqno)
    ssthresh = max (flightsize/2, 2 MSS);         // flightsize=lastseqno-ack_no+1)
    cwnd = ssthresh + 0 MSS;                      // we don't need "+3MSS" as…)
    Reset RETX timer;                             // …pipe is updated using SACK)
    RETX the missing segment;
    pipe = lastseqno-lastackno+1;                 // # of packets in flight & not ACKed
    deflate pipe using correct RXs thanks to SACK;
    while (pipe<cwnd) {
        if (!SACK_list_empty) TX_first_missing_packet();
        else TX_new_packet();
        pipe++;
    }
Every time a new ACK (duplicate or not) arrives:
    pipe  = pipe – nacked;                        // deflate pipe using correct RXs (SACK)
    while (pipe<cwnd) {
        if (!SACK_list_empty) TX_first_missing_packet();
        else TX_new_packet();
        pipe++;
    }
When an ACK that acknowledges new data arrives (ack_no>recover):
    Exit_Fast_Recovery();
```

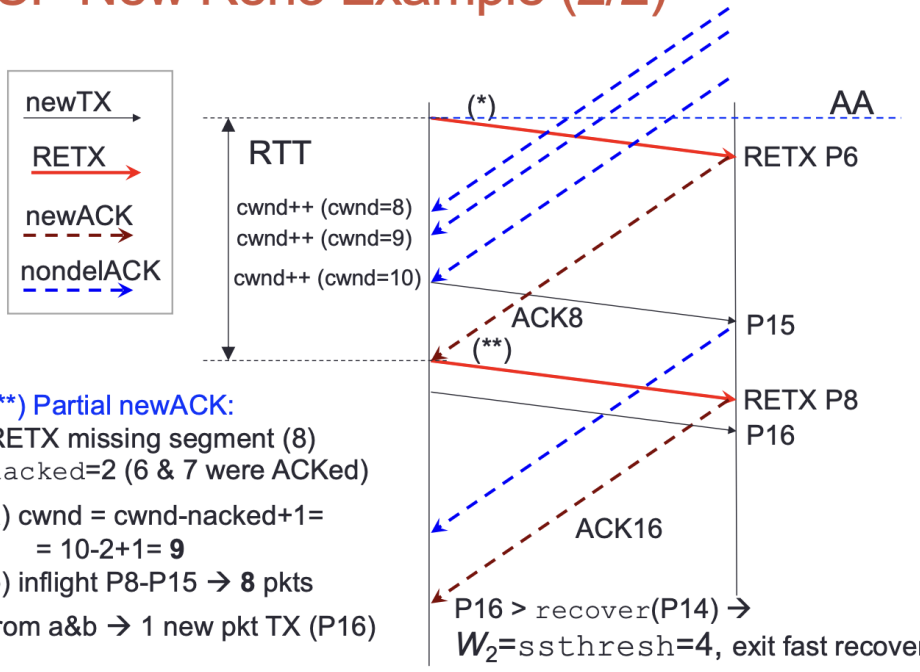
Pseudocode

TCP New Reno Example (1/2)



(*) Enter Fast RETX:
`recover`=P14
`flightsize`=9 (P14-P6+1)
`sshthresh=flightsize/2`=4
`cwnd=sshthresh`+3=7
Reset RETX timer
RETX P6

TCP New Reno Example (2/2)



(**) Partial newACK:
RETX missing segment (8)
`nacked`=2 (6 & 7 were ACKed)

a) cwnd = cwnd-nacked+1=
= 10-2+1= **9**
b) inflight P8-P15 → **8** pkts
from a&b → 1 new pkt TX (P16)

Example

2) Network

Assign IP

Hubs don't need IP addresses
Start from lowest

Routing Tables

We construct a routing table in the following way:

Index (not needed)	Name	Destination	Netmask	Next Hop
1	LAN A	30.20.0.176	28	Direct
2	LAN B	40.20.0.0	16	10.0.0.1
3	LAN C	30.20.0.176	28	10.0.0.2
4	Default Gw	0.0.0.0	0	10.0.0.2

1. This LAN is directly connected to the router
2. This is a common configuration
3. **CROSSOUT:** in this case the next hop=default gw hop and we can delete it
4. Default gateway is always the last entry.

Remark.

- Avoid slow links
- If a router is then directly connected to one host we can omit DTS+MASK in the routing table

How Packets Are Routed

Som exercises don't require the building of routing tables, but already provide one and also the interface config and ask you how some packets are routed.

Routing table

Network	Netmask	Next Hop
131.175.70.0	255.255.254.0	131.175.21.133
131.175.71.128	255.255.255.128	131.175.21.145
131.175.72.0	255.255.254.0	131.175.20.5
131.175.75.192	255.255.255.192	131.175.20.250
0.0.0.0	0.0.0.0	131.175.20.221

Interface configuration

Interface	IP Address	Netmask	MTU [B]
eth0	131.175.21.254	255.255.255.128	1500
eth1	131.175.20.126	255.255.255.128	1000
eth2	131.175.20.132	255.255.255.128	1200

Exercise Example

There are some easy steps to follow:

- Find the network addresses + range of the IP Addresses in the Interface Config table
- For all entries in the Routing table write the range of IPs, the netmask in decimal and finally what interface is associated to the next hop
  Now for every packet to route do the following:
- Check if destination is IP address of one interface. This is **delivered to upper layers**
- Check if the destination is in the interface config, if it is it is **directly forwarded**
- Otherwise check in the routing table by starting with the **biggest netmask**. If no network matches the default gateway is used. In this case the packet is **indirectly forwarded**
- Check what interface the packet is sent from and where it is routed to. If it is sent to the same interface it comes from then no forwarding is done

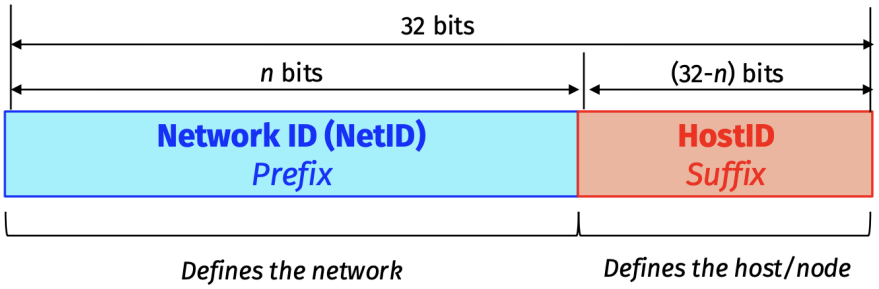
- Check what interface it is forwarded to to see the MTU
  - If Packet Size $< MTU$ nothing happens
  - If Packet Size $> MTU$ the packet must be fragmented
    - If **Don't Fragment Bit = 1** the packet is dropped
    - If **Don't Fragment Bit = 0** the packet is fragmented in $\lfloor \frac{\text{Packet Size}}{MTU} \rfloor$ complete fragments + one smaller last fragment, this is $\lceil \frac{\text{Packet Size}}{MTU} \rceil$ fragments.

2.2) Theory

Ipv4 is a **32-bit (2^{4+1}) address** (with a total of 2^{32} addresses) that addresses an unique connection to the internet. It is universal since it is accepted by any host

IPv4 is **hierarchical** as it is divided into two parts:

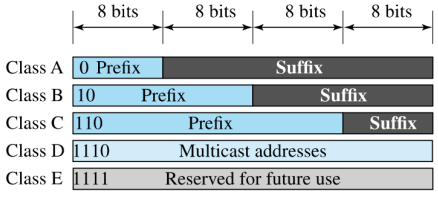
- NetID: prefix of length n and defines the network
- HostID: suffix of length $32 - n$ and defines the host in the network



IPv4

> Nodes on the same network have same NetID and different HostID. Nodes in different networks have different NetID and can have same HostID.

Originally IP were thought as **classfull**, that is there were 3 fixed lengths (8, 16, 24) with 5 spaces (A, B, C, D, E) that had a fixed prefix



Class	Prefixes	First byte
A	n = 8 bits	0 to 127
B	n = 16 bits	128 to 191
C	n = 24 bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

Classfull

This approach has many flaws, therefore we resort to **classless** addressing, where n has a variable length.

Class	First bits	# net bits	# node bits	# networks	# nodes	from	to
A	0	7+1	$32-(7+1)=24$	2^7-2	$2^{24}-2$	0.0.0.0	127.255.255.255
B	10	14+2	$32-(14+2)=16$	2^14	$2^{16}-2$	128.0.0.0	191.255.255.255
C	110	21+3	$32-(21+3)=8$	2^21	2^8-2	192.0.0.0	223.255.255.255

Where the -2 in the # nodes tab is for the broadcast/network addresses and the -2 in the A class # networks are the two reserved network addresses

> **Remark** (How to obtain addresses)**.**

The start/finish addresses are done by:

- start: all bits (besides class) to 0
- finish: all bits (besides class) to 1

Network address:

- All HostID bits to 0

Broadcast address:

- All HostID bits to 1

Classless InterDomain Routing (CIDR) & Netmask

Classless addressing sets an arbitrary value for n, therefore we need to define a new way to identify a network.

Definition 1 (Classless InterDomain Routing)**.**

The slash notation represent the **Classless InterDomain Routing** (CIDR) representation:

$$1x8.1x8.1x8.1x8/n,\ n \in [0, 32] \in \mathbb{N}$$

	Operation
Number of Addresses (N)	$N = 2^{32-n}$
Number of Hosts	$N - 2 = 2^{32-n} - 2$
Network Address	last $32 - n$ bits $= 0 \iff$ HostID bits to 0
Broadcast Address	first $32 - n$ bits $= 1 \iff$ HostID bits to 1

Definition 2 (Netmask)**.**

Given an address and a value for n we define the **netmask** as a 32-bit sequence starting with n ones and the rest are all zeroes
Example:

$$192.168.1.7/27 \rightarrow \text{netmask: } 1x8.1x8.1x8.11100000$$

This allows to find the related info with bit-wise operations (NOT, AND, OR)

	Operation
Number of Addresses	**NOT**(mask)+1
Number of Hosts	**NOT**(mask)-1
Network Address	Any address in block **AND** mask
Broadcast Address	Any address in block **OR NOT**(mask)

Example.

Suppose we have the ip:

$$192.168.1.7/27 \iff 11000000.10101000.00000001.00000111/27$$

CIDR:

- *Number of Addresses* = $2^{32-27} = 32$
- *Network (first) Address:*

$$32 - 27 = 5 \to 11000000.10101000.00000001.00000000/27 \to 192.168.1.0/27$$

- *Broadcast (last) Address:*

$$32 - 27 = 5 \to 11000000.10101000.00000001.00011111/27 \to 192.168.1.31/27$$

- *From here we can see that we have exactly 32 addresses (from .0 to .31)*

Netmask:

- *Netmask:*

$$11111111.11111111.11111111.11100000 \to 255.255.255.112$$

- *Number of Addresses:*

$$00000000.0000000.00000000.0001111 + 1 = 10000 \overset{\text{dec}}{=} 32$$

- *Network Address:*

$$\begin{array}{rllll} & 11111111 & 11111111 & 11111111 & 11100000 \\ & 11000000 & 10101000 & 00000001 & 00000111 \\ \text{AND:} & 11000000 & 10101000 & 00000001 & 00000000 \end{array} \to 192.168.1.0/27$$

- *Broadcast Address: (OR)*

$$\begin{array}{rllll} & 00000000 & 00000000 & 00000000 & 00011111 \\ & 11000000 & 10101000 & 00000001 & 00000111 \\ \text{OR:} & 11000000 & 10101000 & 00000001 & 00011111 \end{array} \to 192.168.1.31/27$$

- *This is the same as before*

Here is a quick reference table with some useful numbers

Binary	Decimal
11111111	255
11111110	254
11111100	252
11111000	248
11110000	240
11100000	224
11000000	192
10000000	128

The association that assigns blocks of addresses to an ISP is called **Internet Corporation for Assigned Names and Numbers (ICANN)** and has to follow these rules:

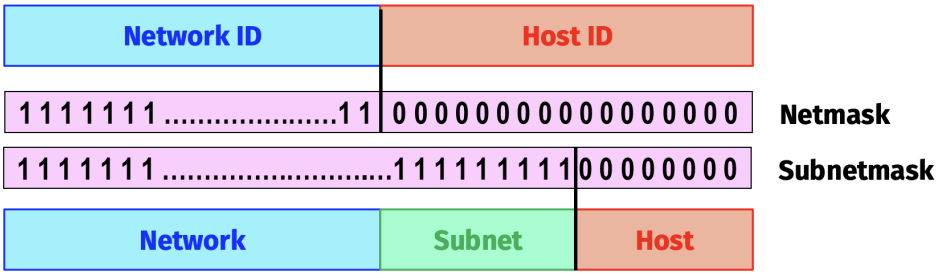
- The requested number of addresses (M) must be a power of 2
- The prefix length of each block (n) is then $n = 32 - \log_2 M$
- The network address of the block will have $\log_2 M$ zeroes

Remark.

If we need k addresses we need to ask for $M = 2^{\lceil \log_2 k \rceil} \implies n = 32 - \lceil \log_2 k \rceil$

Subnetting

A large network can be divided into further subnetworks by creating a subnetmask



Example

Here the initial HostID is divided into SubID + (new) HostID. From here the same rules as before work. Let's call n the netmask, $m \leq 32 - n$ the subnetmask, then:

- HostID has length $32 - n - m$
- Number of subnetworks: 2^m
- Number of Addresses per subnetwork: 2^{32-m-n}

In real world scenarios both a class and a netmask are assigned. For subnetting the netmask > class NetID bits. The class NetID bits can't be changed and thus **the number of bits m for the subnet is given by netmask-class HostID bits**.

Subnetting is achieved by having a netmask different from the bits of the class NetID

The following steps are used fro a correct subnetting:

- N_{sub} must be a power of 2
- each network has a prefix of $n = 32 - \log_2 N_{sub}$
- The starting address in subnetwork should be divisible by the number of addresses in the subnetwork → we need to start the process by assigning first addresses to larger subnetworks. ???

here are some useful examples:

Example.

Suppose to have a class B (n=16) address with netmask of $255.255.255.0 = 24$. *Then we have:*

- $32 - 24 = 8$ *HostID bits*

- $24 - 16 = 8$ *SubnetID bits* $\implies 2^8 = 256$ *addresses*

Suppose to have the following IP: $147.162.8.3/24$ *we can find the SubID:*

- *Rewrite the IP:* $147.162.8.3 \to 10010011.10100010.00001000.00000011$
- *We know that the first 16 bits are used for the NetID, and the following 8 are the SubID*
- *The SubID is* $00001000 \overset{\text{dec}}{=} 8$ *and the HostID is* $00000011 \overset{\text{dec}}{=} 3$

Example.

Create subnets with at least 500 hosts with network address $132.78.0.0$ *and NetID of 16 bits:*

- *Since 500 is not a power of 2 we find the closest bigger power* $\to \log_2 500 = 9 \to 2^9 = 512$
- *Then we have HostID=9 and thus* $n = \text{NetID} + m = 32 - \text{HostID} = 23 \to m = 7$
- *We subnetted to have 128 subnets with 510 hosts each*

Example.

Create at least 1000 subnets with Network address $128.234.0.0$ *and NetID 16:*

- *1000 is not a power of 2* $\to m = \log_2 1000 = 10 \to 1024$ *subnets*
- *Then we have that* $n = \text{NetID} + m = 32 - \text{HostID} = 26 \to \text{HostID} = 6$
- *We have created 1024 subnets with 62 hosts each*

Example (Uneven Subnetting)**.**

We have the addresses $14.27.74.0/24$ *and want to split them in 3 subnetworks of type:*

1. *one subblock of* $10 \to 16$ *addresses* $\implies \text{HostID}_1 = 4$
2. *one of* $60 \to 64 \implies \text{HostID}_2 = 6$
3. *one of* $120 \to 128 \implies \text{HostID}_3 = 7$

Since we already calculated the HostID in the 3 cases we already know that $\text{HostID}_{\max} 32 - 24 = 8$ *satisfies our requirements. We use rule 3* $\to$ *we start with the biggest subnetwork:*

- $n_3 = 32 - \text{HostID}_3 = 25$ *and thus the network address is* $14.27.74.0/25$ *and broadcast* $14.27.74.0.01111111/25 \to 14.27.74.127/25$ *(128-1 difference)*
- $n_2 = 26$. *Notice that we can't use net address of* $14.27.74.0/26$ *since these addresses are already taken by subnet 3. All addresses until* $14.27.74.128$ *are taken, these relate to SubID* $00, 01$ *so we choose SubID=*10 *and we have net/broad addresses* $14.27.74.128/26$ *and* $14.27.74.191/26$ *(64-1 difference)*
- $n_1 = 28$. *As before the SubIDs starting with 0 or 10 are already taken, so we use* 1100. *We have net/broad addresses* $14.27.74.192/28$ *and* $14.27.74.207/28$ *(16-1 difference)*

From the last exercise we can derive the following important remark:

Remark (No overlapping)**.**

Subnets can't have blocks that overlap. Since the IP datagram doesn't contain the netmask it will cause confusion to the routing.

Moreover when deciding the network address of a block it must start with an address with HostID=0. If all the addresses from the network to the broadcast address (HostID=1) aren' already occupied the block can be used for subnetting

From here another remark can be made:

Remark 3 (Constraints on Network and Broadcast Addresses)**.**

The final conclusion is that a network address must have HostID=0 and the broadcast address must have HostID=1. If this is not the case the given address is not a network/broadcast address or the network is badly set up.

Supernetting

Class A, B networks are running out, therefore we can group many class C networks to join them into a bigger network

Example.

We have the following blocks

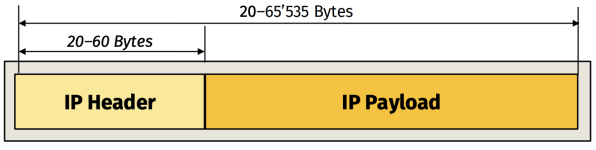
$$193.23.136.0/24$$
$$193.23.137.0/24$$
$$193.23.138.0/24$$
$$193.23.139.0/24$$

All these go from $.0$ *to* $.255$ *so the full range from* $193.23.136.0/24$ *to* $192.23.139.255/24$ *are covered. These are* $256 \cdot 4 = 2^{10} = 1024$ *addresses, exactly the number of addresses of a /22 network. From here it is clear that these can be grouped into the single block*

$$193.23.136.0/22$$

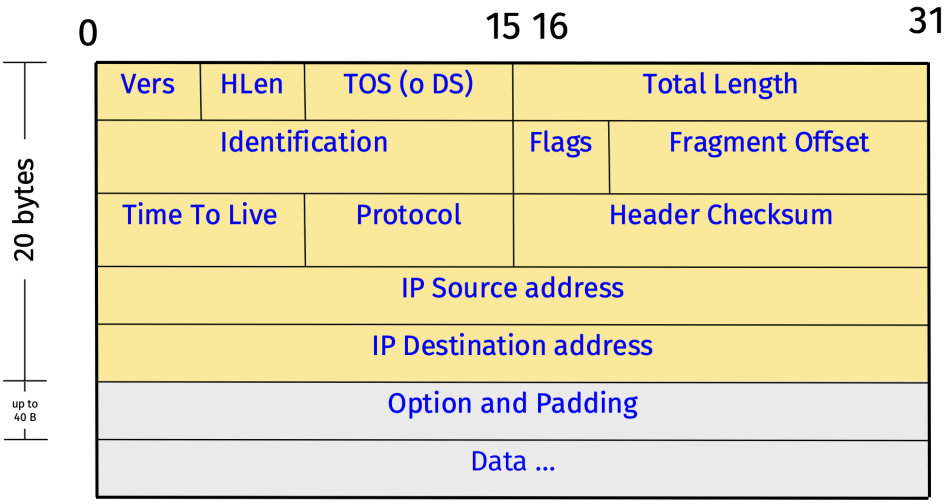
Datagram

Here is the datagram of the IP in detail:



IP Datagram

Let's focus on the header:

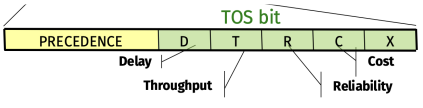


IP Datagram Header

Name	Length (Bits)	Description	More in depth
Vers	4	version of protocol used	clearly always 0100
HLeng	4	stores the total length of the header in *bytes*	The minimum is defined as $0101\|\|_2 = 5\|\|_{10} \rightarrow 5 \cdot 4 = 20$ Bytes. The maximum is $1111\|\|_2 = 15\|\|_{10} \rightarrow 15 \cdot 4 = 60$ Bytes
Type Of Service (TOS)	8	defines how datagram should be handled.	See below
Total Length	8	length of header + data in *bytes*	Length of data can be found by data=Total Length - HLeng
Time To Live (TTL)	8	maximum # of hops before getting dropped.	On drop an ICMP may be sent
Protocol	8	Upper layer protocol used	TCP = 06, UDP=17
Checksum	16	rough form of error detection	covers only header
IP Source/Destination	32x2	IP Address of Source/Destination	These can't be changed while in flight. Netmasks are NOT included. Routing tables should handle this
Identification	16	helps destination in reassembling datagram (all fragments with same flag should be assembled into one datagram)	Same for all fragments
Flags	3	Fragmentation control	R = reserved. D = don't fragmen (if bigger than MTU send error message). M = More Fragment, this is not the last fragment
Offset	13	Relative position of fragment with respect to the whole datagram	It is the offset of the data in the og datagram measured in units of 8 bytes (first bit/8)
Options/padding	up to 40	used for testing and debugging	

Remark (TOS in detail)**.**

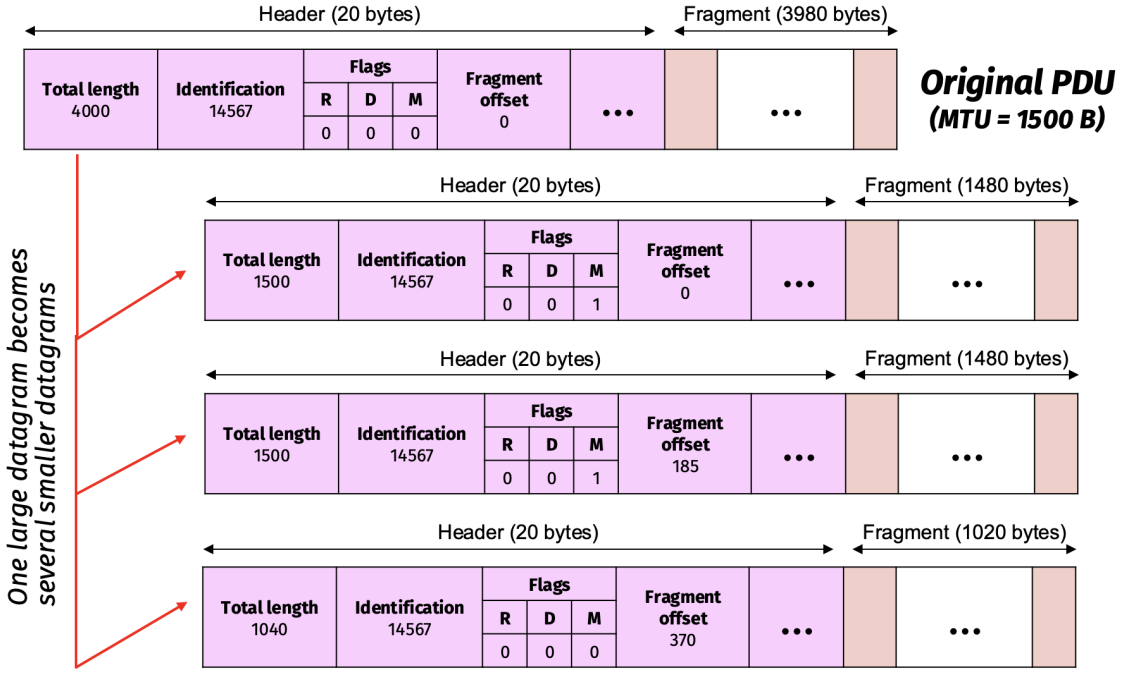
Here are some more details on the TOS



TOS

TOS Bits	*Description*
0000	Normal (default)
0001	Minimize cost
0010	Maximize reliability
0100	Maximize throughput
1000	Minimize delay

TOS Bits



Fragmentation Example

The biggest problem with fragmentation is that the loss of one fragment means to lose the entire datagram. To avoid this the IP layer shouldn't exceed the MTU of the network(s) it will go to

Fragmentation **changes** the following fields:

- **Length, Flags, Offset, Header Checksum**
  And doesn't change:
- Version, TOS, Id, TTL, Protocol, IP src/dst

Dec	Hex	Bin	Dec	Hex	Bin	Dec	Hex	Bin	Dec	Hex	Bin
0	0	00000000	64	40	01000000	128	80	10000000	192	c0	11000000
1	1	00000001	65	41	01000001	129	81	10000001	193	c1	11000001
2	2	00000010	66	42	01000010	130	82	10000010	194	c2	11000010
3	3	00000011	67	43	01000011	131	83	10000011	195	c3	11000011
4	4	00000100	68	44	01000100	132	84	10000100	196	c4	11000100
5	5	00000101	69	45	01000101	133	85	10000101	197	c5	11000101
6	6	00000110	70	46	01000110	134	86	10000110	198	c6	11000110
7	7	00000111	71	47	01000111	135	87	10000111	199	c7	11000111
8	8	00001000	72	48	01001000	136	88	10001000	200	c8	11001000
9	9	00001001	73	49	01001001	137	89	10001001	201	c9	11001001
10	a	00001010	74	4a	01001010	138	8a	10001010	202	ca	11001010
11	b	00001011	75	4b	01001011	139	8b	10001011	203	cb	11001011
12	c	00001100	76	4c	01001100	140	8c	10001100	204	cc	11001100
13	d	00001101	77	4d	01001101	141	8d	10001101	205	cd	11001101
14	e	00001110	78	4e	01001110	142	8e	10001110	206	ce	11001110
15	f	00001111	79	4f	01001111	143	8f	10001111	207	cf	11001111
16	10	00010000	80	50	01010000	144	90	10010000	208	d0	11010000
17	11	00010001	81	51	01010001	145	91	10010001	209	d1	11010001
18	12	00010010	82	52	01010010	146	92	10010010	210	d2	11010010
19	13	00010011	83	53	01010011	147	93	10010011	211	d3	11010011
20	14	00010100	84	54	01010100	148	94	10010100	212	d4	11010100
21	15	00010101	85	55	01010101	149	95	10010101	213	d5	11010101
22	16	00010110	86	56	01010110	150	96	10010110	214	d6	11010110
23	17	00010111	87	57	01010111	151	97	10010111	215	d7	11010111
24	18	00011000	88	58	01011000	152	98	10011000	216	d8	11011000
25	19	00011001	89	59	01011001	153	99	10011001	217	d9	11011001
26	1a	00011010	90	5a	01011010	154	9a	10011010	218	da	11011010
27	1b	00011011	91	5b	01011011	155	9b	10011011	219	db	11011011
28	1c	00011100	92	5c	01011100	156	9c	10011100	220	dc	11011100
29	1d	00011101	93	5d	01011101	157	9d	10011101	221	dd	11011101
30	1e	00011110	94	5e	01011110	158	9e	10011110	222	de	11011110
31	1f	00011111	95	5f	01011111	159	9f	10011111	223	df	11011111
32	20	00100000	96	60	01100000	160	a0	10100000	224	e0	11100000
33	21	00100001	97	61	01100001	161	a1	10100001	225	e1	11100001
34	22	00100010	98	62	01100010	162	a2	10100010	226	e2	11100010
35	23	00100011	99	63	01100011	163	a3	10100011	227	e3	11100011
36	24	00100100	100	64	01100100	164	a4	10100100	228	e4	11100100
37	25	00100101	101	65	01100101	165	a5	10100101	229	e5	11100101
38	26	00100110	102	66	01100110	166	a6	10100110	230	e6	11100110
39	27	00100111	103	67	01100111	167	a7	10100111	231	e7	11100111
40	28	00101000	104	68	01101000	168	a8	10101000	232	e8	11101000
41	29	00101001	105	69	01101001	169	a9	10101001	233	e9	11101001
42	2a	00101010	106	6a	01101010	170	aa	10101010	234	ea	11101010
43	2b	00101011	107	6b	01101011	171	ab	10101011	235	eb	11101011
44	2c	00101100	108	6c	01101100	172	ac	10101100	236	ec	11101100
45	2d	00101101	109	6d	01101101	173	ad	10101101	237	ed	11101101
46	2e	00101110	110	6e	01101110	174	ae	10101110	238	ee	11101110
47	2f	00101111	111	6f	01101111	175	af	10101111	239	ef	11101111
48	30	00110000	112	70	01110000	176	b0	10110000	240	f0	11110000
49	31	00110001	113	71	01110001	177	b1	10110001	241	f1	11110001
50	32	00110010	114	72	01110010	178	b2	10110010	242	f2	11110010
51	33	00110011	115	73	01110011	179	b3	10110011	243	f3	11110011
52	34	00110100	116	74	01110100	180	b4	10110100	244	f4	11110100
53	35	00110101	117	75	01110101	181	b5	10110101	245	f5	11110101
54	36	00110110	118	76	01110110	182	b6	10110110	246	f6	11110110
55	37	00110111	119	77	01110111	183	b7	10110111	247	f7	11110111
56	38	00111000	120	78	01111000	184	b8	10111000	248	f8	11111000
57	39	00111001	121	79	01111001	185	b9	10111001	249	f9	11111001
58	3a	00111010	122	7a	01111010	186	ba	10111010	250	fa	11111010
59	3b	00111011	123	7b	01111011	187	bb	10111011	251	fb	11111011
60	3c	00111100	124	7c	01111100	188	bc	10111100	252	fc	11111100
61	3d	00111101	125	7d	01111101	189	bd	10111101	253	fd	11111101
62	3e	00111110	126	7e	01111110	190	be	10111110	254	fe	11111110
63	3f	00111111	127	7f	01111111	191	bf	10111111	255	ff	11111111

Table